01_m0_m2

M0 - Foundation & Toolchain

Objective

Stand up the empty Helios repository so that dbt can authenticate to BigQuery, run a bounded smoke query over the GA4 public sample, and report all-green on `dbt debug`. Nothing transforms data yet; M0 only proves the toolchain (Python venv, GCP/IAM/ADC, dbt config, git) is wired correctly and cheaply. This is the foundation every later milestone builds on, so the exit gate is hard: `dbt debug` all-green, ADC authenticates, and a `_TABLE_SUFFIX`-bounded query over `events_*` returns rows under the byte budget.

Files to create

Path	Copy from
`requirements.txt`	exact package list below (StackToolchain)
`.gitignore`	snippet below
`dbt_project.yml`	DBT_GUIDE.md section 1 ("dbt_project.yml") - paste verbatim
`profiles.yml`	DBT_GUIDE.md section 1 ("profiles.yml") - paste verbatim; set `project:` to your GCP project id
`packages.yml`	DBT_GUIDE.md section 1 ("packages.yml") - paste verbatim
`mcp_servers.yaml`	stub only (a single `servers:` key with a TODO comment); finalized in M6
`docs/CLAUDE.md`	ALREADY EXISTS - do not regenerate

`requirements.txt` (exact pins for the whole project; M0 only needs the dbt + google rows but pin the rest now so the venv is stable):

```
dbt-core≥1.7,<1.9
dbt-bigquery≥1.7,<1.9
google-cloud-bigquery≥3.17
scipy≥1.11
statsmodels≥0.14
prophet≥1.1
pmdarima≥2.0
```

```
pandas≥2.1
pyyaml≥6.0
mcp≥1.0
anthropic≥0.39
pytest≥8.0
```

`.gitignore` (keep secrets and build artifacts out of git):

```
.venv/
target/
dbt_packages/
logs/
*.json            # service-account keys must never be committed
!eval/baselines/*.json
.env
__pycache__/
.user.yml
```

Commands to run

```
# 1. Repo + venv (Windows activation shown; POSIX = source .venv/bin/activate)
git init
python -m venv .venv
.venv\Scripts\activate            # Windows;  POSIX: source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt

# 2. GCP auth via ADC (interactive OAuth for dev - no key files on the laptop)
gcloud auth application-default login
gcloud config set project <PROJECT>          # e.g. helios-analytics

# 3. Least-privilege service account (used by warehouse-mcp / CI later; NOT Owner)
gcloud iam service-accounts create helios-wh \
  --display-name "Helios warehouse read-only"
gcloud projects add-iam-policy-binding <PROJECT> \
  --member "serviceAccount:helios-wh@<PROJECT>.iam.gserviceaccount.com" \
  --role roles/bigquery.dataViewer
gcloud projects add-iam-policy-binding <PROJECT> \
  --member "serviceAccount:helios-wh@<PROJECT>.iam.gserviceaccount.com" \
  --role roles/bigquery.jobUser
# For a server/CI key (only when you need a JSON key; dev uses OAuth ADC above):
#   gcloud iam service-accounts keys create helios-wh.json \
#     --iam-account helios-wh@<PROJECT>.iam.gserviceaccount.com
#   export GOOGLE_APPLICATION_CREDENTIALS=$PWD/helios-wh.json   # never commit this

# 4. dbt: install pinned packages, then prove the connection
dbt deps
dbt debug

# 5. Bounded smoke query over events_* (prune shards with _TABLE_SUFFIX!)
```

```
bq query --use_legacy_sql=false --maximum_bytes_billed=1073741824 \
'SELECT COUNT(*) AS events, COUNT(DISTINCT user_pseudo_id) AS users
 FROM `bigquery-public-data.ga4_obfuscated_sample_ecommerce.events_*`
 WHERE _TABLE_SUFFIX BETWEEN "20210101" AND "20210107"'
```

Tests to run

```
dbt debug          # asserts: profiles.yml parses, project is valid, BigQuery connection OK
dbt deps           # asserts: all four packages resolve and install into dbt_packages/
gcloud auth application-default print-access-token   # asserts: ADC token is mintable
```

The `bq` smoke query asserts that ADC can read the public dataset, that the `US` location resolves, and - critically - that pruning seven shards keeps the scan far under the 1 GiB cap passed via `--maximum_bytes_billed`. A useful sanity check: a single `events_YYYYMMDD` shard in this sample is on the order of tens of MB, so seven shards must bill far below 1 GiB; if `bq` reports it would scan gigabytes, the `_TABLE_SUFFIX` filter is not being applied (often because it was placed in a subquery the optimizer cannot prune, or the wildcard table name was mistyped).

Success criteria

- `dbt debug` prints `All checks passed!` (connection, profile, project all green).
- `gcloud auth application-default print-access-token` returns a token (ADC works).
- The smoke query returns a non-zero `events` count and bills well under 1 GiB (visible in the job stats), proving shard pruning works.
- `mcp_servers.yaml` exists as a stub; `git status` shows no `*.json` key staged.

Common mistakes

- IAM: granting `roles/owner` (or `roles/editor`) instead of least-privilege `roles/bigquery.dataViewer` + `roles/bigquery.jobUser`. Fix: bind exactly those two roles; `jobUser` is required to *run* a query, `dataViewer` only to *read* a table.
- Forgetting ADC entirely: dbt fails with "Could not automatically determine credentials." Fix: run `gcloud auth application-default login`, or set `GOOGLE_APPLICATION_CREDENTIALS` to the SA key path.
- Location mismatch: leaving `location:` unset or set to a region (`us-central1`). The GA4 public sample lives in the `US` multi-region, so any other value yields "Not found: Dataset ... in location ...". Fix: `location: US` in both `profiles.yml` targets.
- Cost blowout: running the smoke query without a `_TABLE_SUFFIX` filter scans all 90+ shards (full history). Fix: always bound `_TABLE_SUFFIX` and pass `--maximum_bytes_billed`; `profiles.yml` also sets the 5 GiB per-query cap.
- Committing the SA JSON key: leaks credentials into git history. Fix: `.gitignore` excludes `*.json`; dev should use OAuth ADC and avoid keys altogether.

- Too many threads in `profiles.yml` against a small dev quota causes job-queue thrash. Keep dev `threads: 8` as pinned.
- Omitting `maximum_bytes_billed` in `profiles.yml`: a careless later query then silently scans (and bills) the whole dataset instead of failing fast. Fix: keep the 5 GiB dev cap / 20 GiB prod ceiling from the pinned `profiles.yml`; this cap is the project's per-run byte budget and the primary cost guardrail.
- Pointing `profile:` in `dbt_project.yml` at a name that does not match the top-level key in `profiles.yml`: `dbt debug` fails with "Could not find profile named 'helios'." Fix: both must read `helios`.

M1 - Sources, Macros & Seeds

Objective

Declare the GA4 source contract and create the four shared abstractions every downstream layer reuses exactly once: the typed param extractor, the canonical session-key, the single-source-of-truth channel grouping, and the revenue reconciliation test - plus the seed that backs channel mapping. After M1 the project has no models yet, but `dbt deps` + `dbt seed` succeed and every macro compiles. These macros are load-bearing: putting channel logic or the session-key expression in more than one place is the root cause of distinct-count drift and an inventable 11th channel group.

Files to create

Path	Copy from
`models/staging/src_ga4.yml`	DBT_GUIDE.md section 3.1 (and the fuller version in section 2, "sources.yml for src_ga4") - paste verbatim; set `database` / `schema` to `bigquery-public-data` / `ga4_obfuscated_sample_ecommerce`
`macros/get_event_param.sql`	DBT_GUIDE.md section 3.4 ("get_event_param") - paste verbatim
`macros/sessionize.sql`	DBT_GUIDE.md section 3.4 ("sessionize") - paste verbatim
`macros/channel_group.sql`	DBT_GUIDE.md section 3.4 ("channel_group / channel_group_case") - paste verbatim; the file defines BOTH `channel_group()` and `channel_group_case()`
`macros/test_revenue_reconciles.sql`	DBT_GUIDE.md section 6 (custom generic test) - the `revenue_reconciles` mart-total == source-total generic

Path	Copy from
	test
`seeds/channel_group_mapping.csv`	canonical `source,medium,channel_group` rows (header below); feeds `channel_group_case`

The seed is a small CSV with a header row `source,medium,channel_group` and one row per known mapping; it must only ever resolve to the canonical 10 groups (`Direct, Organic Search, Paid Search, Display, Paid Social, Organic Social, Email, Affiliates, Referral, Other`). Its `+column_types` (all `string`) are already declared in `dbt_project.yml` under `seeds:` .

`src_ga4.yml` carries the source contract: `database: bigquery-public-data` , `schema: ga4_obfuscated_sample_ecommerce` , the `events` table with `identifier: 'events_*'` , the production freshness SLA (`warn_after: 36h` , `error_after: 48h`), and `not_null` tests on `event_date` , `event_timestamp` , `event_name` , `user_pseudo_id` . On the static sample, freshness is informational only - keep the `dbt_utils.recency` source test at `severity: warn` so a frozen sample does not hard-fail every build.

Commands to run

```
.venv\Scripts\activate              # POSIX: source .venv/bin/activate
dbt deps                            # (re)install packages if not already present
dbt seed                            # loads channel_group_mapping.csv into the dev dataset
dbt parse                           # compiles the project graph; surfaces macro syntax errors
# Prove each macro compiles by rendering it in a throwaway compile:
dbt compile --inline \
  "select {{ sessionize('user_pseudo_id','ga_session_id') }} as session_key"
dbt compile --inline \
  "select {{ channel_group_case('source','medium','gclid') }} as channel_group"
```

Tests to run

```
dbt seed                # asserts: the CSV loads; column types match dbt_project.yml seeds: block
dbt parse               # asserts: src_ga4.yml is valid YAML and all macros are syntactically
        valid Jinja+SQL
dbt compile --inline "select {{ get_event_param('ga_session_id','int') }}"   # asserts: macro
        renders the correlated UNNEST subquery
```

A passing `dbt parse` proves the source declaration and the macro definitions are well-formed; the inline `dbt compile` calls prove each macro renders to valid BigQuery SQL (the real golden-value unit tests on `sessionize` / `channel_group_case` land alongside the M3 keystones).

Success criteria

- `dbt deps` and `dbt seed` both exit 0; `channel_group_mapping` appears as a table in `helios_dev`.
- `dbt parse` succeeds with no compilation errors.
- `sessionize()` renders exactly `to_hex(md5(concat(user_pseudo_id, '-', cast(ga_session_id as string))))` - the one canonical expression, nowhere else.
- `channel_group_case()` renders a CASE producing exactly the 10 canonical groups and is `gclid`-aware (a non-null `gclid` forces Paid Search).
- The seed's `channel_group` column contains no value outside the 10-group set.

Common mistakes

- Pinning packages loosely or not at all: a `dbt_utils` major bump breaks `generate_surrogate_key` / `accepted_range`. Fix: keep the version ranges from `packages.yml` (e.g. `dbt_utils ≥1.1.0,<2.0.0`) and re-run `dbt deps`.
- Duplicating channel logic: writing a second `CASE` that buckets channels anywhere other than `channel_group_case()`. Fix: every model and the semantic layer must call the one macro; there is no inline channel CASE.
- Inventing an 11th group (e.g. "Paid Other") in the macro or seed. Fix: the only allowed outputs are the 10 canonical groups; anything else is a hard error in the M3/M4 `accepted_values` test.
- Registry filename drift: later wiring `mcp_servers.yaml` at `semantic_models.yml` when the live file is `semantic_layer.yaml`. Not an M1 file, but note it now - the macro/seed conventions feed that registry.
- Source filename drift: `_ga4__sources.yml` vs `src_ga4.yml`. Either name works as long as staging refs `source('src_ga4', 'events')`; pick one and keep `name: src_ga4` inside it. The pinned milestone path is `src_ga4.yml`.
- Treating freshness as a hard gate on the sample: the newest shard is `events_20210131`, so a 48h `error_after` trips forever. Fix: keep freshness informational (`severity: warn`) on dev/sample; the `prod` target enables the `dbt source freshness && dbt build` gate.
- Forgetting the seed `+column_types`: BigQuery may infer the wrong type for an all-numeric `source` value. The `dbt_project.yml` `seeds:` block already pins all three columns to `string` - do not remove it.

M2 - Staging Models

Objective

Build the renaming layer: two `view` staging models that are pure 1:1 projections of the GA4 source - rename to snake_case, cast types, parse `_TABLE_SUFFIX` into a real `event_date`, lightly

flatten the scalar structs (`device.*` , `geo.*` , `ecommerce.*`), and unnest `event_params` once. Staging does no joins, no aggregations, no dedup, and no `SELECT *` against the source. After M2, `dbt build --select staging` passes with key uniqueness/not_null tests green. These two models are the only place in the entire project that touches the raw `event_params` array and the nested structs.

Files to create

Path	Copy from
`models/staging/stg_ga4__events.sql`	DBT_GUIDE.md section 3.2 - paste verbatim (1:1 typed/renamed events; `session_key` via `sessionize()` ; session-scoped `event_params` source/medium plus first-touch fallback)
`models/staging/stg_ga4__event_params.sql`	DBT_GUIDE.md section 3.3 - paste verbatim (long `(event x param key)` table; the only `UNNEST(event_params)` in the repo)
`models/staging/stg_ga4__schema.yml`	DBT_GUIDE.md section 3.5 - paste verbatim (docs + `not_null` / `unique` /range tests)

Key points carried in the code you copy: `stg_ga4__events` selects an explicit column list (never `SELECT *` against the source), computes `event_date = parse_date('%Y%m%d', _table_suffix)` , surfaces `ga_session_id` and the session-scoped `event_source` / `event_medium` / `gclid` via `get_event_param( … )` , keeps `traffic_source.*` only as `first_touch_*` (FALLBACK, not session source), and emits `session_key` exclusively through `sessionize()` . The `not_null` test on `session_key` is conditioned with `where: "ga_session_id is not null"` because GA4 emits a few session-less events (e.g. `first_open`).

Commands to run

```
.venv\Scripts\activate              # POSIX: source .venv/bin/activate
dbt build --select staging          # builds BOTH views AND runs their schema.yml tests in one
        pass
# Inspect the materialized views / sample output:
dbt show --select stg_ga4__events --limit 5
dbt build --select +stg_ga4__event_params   # the model plus its (source) upstream, if needed
```

Tests to run

```
dbt build --select staging          # builds + tests staging in dependency order (do NOT run
        run+test separately)
dbt test  --select stg_ga4__events  # asserts: session_key not_null (where ga_session_id is not
        null),
```

```
                                    #           event_date/event_timestamp/user_pseudo_id
        not_null,
                                    #           purchase_revenue_in_usd ≥ 0
dbt test  --select stg_ga4__event_params   # asserts: param_key/event_timestamp/user_pseudo_id
        not_null
```

`dbt build --select staging` is the canonical command: it materializes the two views and runs every test attached to them in one DAG pass, so a model can never be "built but untested." Running `dbt run` then `dbt test` as separate steps risks shipping an untested staging layer.

Success criteria

- `dbt build --select staging` exits 0; both `stg_ga4__events` and `stg_ga4__event_params` exist as views in `helios_dev`.
- All `not_null` tests pass; `session_key` is non-null for every row with a non-null `ga_session_id`.
- `event_date` is a real `DATE` (not the `YYYYMMDD` string) on every row.
- No staging model contains a literal `bigquery-public-data...` reference - both read through `source('src_ga4', 'events')`.
- The only `UNNEST(event_params)` in the repo lives in `stg_ga4__event_params`.

Common mistakes

- Running `dbt run` and `dbt test` separately instead of `dbt build`: leaves a window where staging is materialized but unverified. Fix: always `dbt build --select staging`.
- `SELECT *` against the source: scans every nested column (cost) and lets a new GA4 export column silently leak into marts. Fix: enumerate every column explicitly, as the copied SQL does; the inner CTE is already explicit before the final `select *`.
- Using event-level `traffic_source.*` as the session source: it is GA4 user first-touch attribution, identical across all of a user's sessions, and mis-credits returning users. Fix: surface session-scoped `event_params.source/medium` as `event_source` / `event_medium` and keep `traffic_source.*` as `first_touch_*` fallback only (resolution happens in the M3 sessionized keystone).
- Not pruning shards: a staging view over `events_*` with no `_TABLE_SUFFIX` bound scans all shards when built. Fix: the prod build narrows via `_TABLE_SUFFIX`/lookback (see the commented incremental note in `stg_ga4__events`); on the static sample the window is the fixed sample range.
- Hand-writing the session key as `FARM_FINGERPRINT(...)` or `COUNT(*)` for sessions: causes distinct-count drift versus the canonical `TO_HEX(MD5(...))`. Fix: only ever emit `session_key` via `sessionize()`; count sessions as `COUNT(DISTINCT session_key)`.
- An unconditional `not_null` on `session_key`: fails on the legitimate session-less events. Fix: keep the `config: {where: "ga_session_id is not null"}` clause from the copied

schema.yml.

- Aggregating or deduping in staging: any `GROUP BY` or filtering of business rows belongs downstream. Fix: staging stays a pure 1:1 projection; sessionization and the funnel logic are the M3 intermediate concern.
- Referencing the source by a hard-coded table name (`from \` bigquery-public-data.ga4_obfuscated_sample_ecommerce.events_*` `) instead of` {{ source('src_ga4', 'events') }} `: breaks lineage and the dev/prod repoint. Fix: always go through` source() `;` dbt_project_evaluator` flags any model that skips it.
- Building staging before M1 is green (macros not compiling, seed not loaded): `stg_ga4__events` calls `sessionize()` and `get_event_param()`, so a missing macro is a hard compile error. Fix: confirm `dbt parse` is clean and `dbt seed` has run before `dbt build --select staging`.